

줄프

연구 개요

연구 제목: OmegaETH: 데이터 가용성(DA) 비용 효율을 고려한 Layer 2 트랜잭션 오더링 최적화

연구 요약

최신 고성능 Layer 2 블록체인은 처리 속도를 극대화하기 위해 트랜잭션을 단순 선착순(FCFS, First-Come First-Served)으로 정렬합니다. 하지만 이 방식은 트랜잭션 데이터를 메인넷이나 별도의 데이터 가용성 레이어(DA Layer)에 저장할 때 발생하는 공간 낭비와 비용 비효율성을 전혀 고려하지 못합니다. 본 연구는 트랜잭션의 데이터 압축률과 할당 공간 충전율을 계산하는 **DA 효율성 점수(DES, DA Efficiency Score)**를 고안하고, 이를 트랜잭션 오더링 스케줄러에 적용하여 실행 속도의 저하를 최소화하면서도 전체 네트워크 유지 비용을 크게 절감할 수 있는 새로운 시퀀싱 알고리즘을 제안합니다.

문제 정의

1. Layer 2 블록체인의 DA 비용 구조

MegaETH와 같은 최신 Layer 2 솔루션은 수만 단위의 TPS(초당 트랜잭션 수)를 달성하기 위해 연산(Execution)은 자체 네트워크에서 수행하고, 최종 데이터 기록(Data Availability)은 EIP-4844로 도입된 Blob 데이터 구조나 별도의 EigenDA를 활용합니다. Blob은 고정된 크기(예: 128KB)를 가지며, 할당된 공간의 실제 사용 여부와 관계없이 전체 비용을 지불해야 하는 구조적 한계가 있습니다.

2. 기존 FCFS 오더링 정책의 한계

현재 대부분의 단일 시퀀서 모델은 들어오는 트랜잭션을 단순한 시간순(FCFS)으로 처리하고 배치(Batch)로 묶어 게시합니다. 이로 인해 다음과 같은 세 가지 비효율성이 발생합니다.

- **공간 미충전(Underfill)에 의한 자원 낭비:** 고정 크기의 Blob에 트랜잭션이 충분히 채워지지 않은 상태에서 제출될 경우 잉여 공간에 대한 가스비(Gas fee)가 낭비됩니다.
- **압축 효율성 무시:** 트랜잭션의 종류(스마트 컨트랙트 호출, 단순 송금 등)에 따라 데이터 압축률이 현저히 다르지만, 현재 시스템은 이를 묶음 구성에 활용하지 못합니다.
- **비용 불균형:** 사용자는 연산 복잡성에 비례하여 수수료를 지불하지만, 네트워크 운영자는 데이터의 누적 크기에 비례하여 비용을 지불하므로 시스템 지속가능성이 저해됩니다.

연구 목표 및 가설

핵심 연구 목표

본 연구는 "트랜잭션 스케줄링 시 데이터 크기와 압축률을 함께 고려하면, 전체 처리량 (Throughput)을 유지하면서도 데이터 저장 비용(DA Cost)을 얼마나 절감할 수 있는가?"를 실증적으로 증명하는 것을 목표로 합니다.

세부 연구 질문 (Research Questions)

- RQ1 (현황 분석):** 기존 FCFS 기반 오더링 시스템에서 발생하는 Blob 공간 낭비 (Underfill rate)와 비용적 손실의 규모는 어느 정도인가?
- RQ2 (성능 개선):** 제안하는 DA 효율성 점수(DES) 알고리즘을 적용할 경우, 기존 대비 동일한 수의 트랜잭션을 처리하는 데 필요한 Blob의 개수(비용)가 얼마나 감소하는가?
- RQ3 (트레이드오프 분석):** 최적화를 위한 재정렬 과정이 개별 트랜잭션의 처리 지연시간(Latency)에 미치는 영향은 어떠하며, 알고리즘 내부의 가중치 조절을 통해 비용과 속도의 최적 균형점을 찾을 수 있는가?

제안 시스템 설계

1. DA Efficiency Score (DES) 기반 오더링 알고리즘

시퀀서의 메모리풀(Mempool)에 대기 중인 트랜잭션들을 평가하여 배치(Batch) 포함 우선 순위를 결정하는 수식(Score)을 설계합니다.

```
# 제안하는 DES 점수 산출 구조 (수도코드 예시)
# DES 값이 높은 트랜잭션부터 현재 생성 중인 블록(배치)에 할당

def compute_DES(tx, current_blob_state):
    # 1. 시급성 점수: 오래 대기한 트랜잭션의 기아 현상(Starvation) 방지
    wait_score = current_time - tx.arrival_time

    # 2. 압축 효율성 점수: 해당 트랜잭션 타입의 통계적 압축률 (높을수록 좁은 공간 차지)
    compress_score = estimate_compressibility(tx.type, tx.data)

    # 3. 공간 적합도 점수: 현재 구성 중인 Blob의 남은 공간(ex: 128KB)에
    # 정확히 들어맞아 낭비를 최소화하는지 여부 평가 (Knapsack Problem 형태)
    fit_score = calculate_space_fitness(tx.estimated_size, current_blob_state.remaining_space)

    return (alpha * wait_score) + (beta * compress_score) + (gamma * fit_score)
```

2. 스케줄러 시뮬레이터 설계

실제 블록체인 코어 레벨을 직접 수정하는 대신, **단독 실행 가능한 트랜잭션 스케줄러 시뮬레이터**를 구축하여 알고리즘의 유효성을 검증합니다.

- **입력(Input):** 실제 이더리움/L2 트랜잭션 데이터셋(타임스탬프, 크기, 타입 등)
- **처리(Process):** FCFS 스케줄러와 DES 기반 스케줄러가 동일한 트랜잭션 큐를 독립적으로 처리
- **출력(Output):** 생성된 Blob의 총 개수, 평균 충전율(Fill rate), 개별 트랜잭션의 처리 대기 시간 통계

관련 연구 및 차별점

관련 선행 연구

- **EIP-4844 (Proto-Danksharding) 분석:** 2024년 이더리움 덴컨 업그레이드로 도입된 128KB 고정 크기의 데이터 저장소. 이 규격으로 인해 '고정 비용 지불 구조'가 생겼으며, 본 연구의 직접적인 배경이 됩니다.
- **"180 Days After EIP-4844" (arXiv:2410.04111):** 실제 이더리움 데이터 분석 결과, 많은 Layer 2가 고정 크기 Blob을 완전히 채우지 못한 채 제출하여 최대 70% 이상의 공간이 낭비되고 있다는 실증적 연구입니다. 이 논문은 여러 롤업이 Blob을 공유하는 방식을 제안했습니다.
- **TimeBoost (Arbitrum, 2025):** 트랜잭션의 도착 시간과 우선순위 수수료(Tip)를 종합적으로 고려하는 최신 오더링 알고리즘입니다.

본 연구의 독창성 (차별점)

기존 연구들은 주로 "다수의 Layer 2 네트워크 간에 Blob을 어떻게 공유할 것인가"와 같이 거시적인 관점에서 접근했습니다. 반면, 본 연구는 **단일 Layer 2 내부의 시퀀서 알고리즘 자체를 개선**하여 근본적으로 데이터 생성 밀도(Density)를 높이는 접근법입니다. 또한 단순히 '빨리 처리하는 것'을 넘어 '**저렴하게 압축하여 담는 것**'을 스케줄링의 새로운 평가 지표로 제안한다는 점에서 차별성을 갖습니다.

연구 방법론

1. 데이터 수집 및 전처리 (Data Collection)

- 이더리움 메인넷 또는 주요 L2(Arbitrum, Optimism 등)의 과거 트랜잭션 데이터를 파싱하여 연구에 사용할 가상 워크로드(Workload) 생성
- 개별 트랜잭션의 크기, 종류(ERC-20, NFT 등), 수수료, 도착 시간 등 핵심 정보 추출

2. 소프트웨어 구현 (Implementation)

- Python (또는 Rust)을 활용하여 독자적인 "**L2 시퀀서 시뮬레이터**" 개발
- **비교군(Baseline) 스케줄러**: 단순 선착순(FCFS) 모델
- **실험군(Proposed) 스케줄러**: 제안하는 DA 효율성 점수(DES) 모델 적용

3. 성능 평가 및 지표 측정 (Evaluation)

동일한 트랜잭션 워크로드를 두 스케줄러에 통과시킨 뒤, 아래의 지표를 정량적으로 비교 분석합니다.

- **DA 비용 절감률**: FCFS 대비 생성된 전체 Blob 개수의 감소 비율
- **데이터 압축 효율**: Blob 당 담긴 평균 실제 데이터(bytes)의 증가량
- **사용자 대기 시간(Latency) 변화**: 최적화로 인해 개별 트랜잭션의 처리가 평균 및 최대 몇 ms/블록 정도 지연되었는지 분석 (Trade-off 시각화)

예상 도전 과제

1. 실제 블록체인 노드 수정의 현실적 한계

- **문제점**: MegaETH나 Optimism 등의 실제 블록체인 코어 소프트웨어(C++, Rust 등으로 작성된 수백만 줄의 코드)의 합의 및 시퀀서 레이어를 학부 연구 기간 내에 분석하고 직접 수정하여 배포하는 것은 물리적으로 불가능에 가깝습니다.
- **해결 방안**: 블록체인 전체를 구현하는 대신, 핵심 연구 대상인 '**트랜잭션 스케줄링 로직**'만을 독립된 **소프트웨어 시뮬레이터로 추상화(Abstraction)**하여 개발합니다. 이는 학부 수준에서 충분히 구현 가능하며, 알고리즘의 우수성을 정량적 데이터로 뽑아내어 포스터 및 논문으로 시각화하기에 가장 적합한 접근입니다.

2. 실시간 연산 오버헤드 (Computational Overhead)

- **문제점**: 10ms 단위로 블록을 생성하는 초고속 환경에서, 매번 모든 트랜잭션의 정확한 압축률을 계산(Zlib, Brotli 등 구동)하여 순서를 매기면 스케줄러 자체가 병목(Bottleneck)이 될 수 있습니다.
- **해결 방안**: 모든 트랜잭션을 실시간으로 압축해보는 대신, 트랜잭션의 타입(Method ID)과 페이로드 길이를 기반으로 **사전 계산된 압축률 룩업 테이블(Lookup Table)**이나 가벼운 추정(Heuristic) 함수를 사용하여 연산 비용을 최소화합니다.

3. 특정 트랜잭션의 무한 대기 현상 (Starvation)

- **문제점**: 압축률이 낮고 크기가 애매한 트랜잭션은 오더링 점수(DES)가 계속 낮게 평가되어 영원히 블록에 포함되지 못할 위험이 있습니다.
- **해결 방안**: DES 공식 내의 '**시급성 점수(대기 시간 비례)**' 가중치를 지수함수 형태로 설계하여, 일정 시간이 지나면 DA 효율이 나쁘더라도 강제로 다음 블록에 포함되도록(Aging Mechanism) 보장합니다.

프로젝트 일정

진행 주차	주요 마일스톤 및 산출물	세부 작업 내용
1~2주차	문헌 조사 및 알고리즘 구체화	EIP-4844 구조 분석, DES(DA Efficiency Score) 수식 초안 작성 및 파라미터 정의
3~4주차	트랜잭션 데이터 확보 및 전처리	Etherscan/BigQuery 등을 통해 10만 건 이상의 트랜잭션 데이터 추출, CSV/JSON 워크로드 변환
5~6주차	베이스라인 시뮬레이터(FCFS) 구현	Python 객체지향 기반으로 단순 큐(Queue) 처리 및 Blob(128KB) 패킹을 수행하는 기본 환경 개발
7~8주차	OmegaETH(DES) 스케줄러 통합	압축률 추정 함수(Heuristic) 구현, 스코어링 기반 우선순위 큐(Priority Queue) 로직 적용
9~10주차	성능 평가 및 파라미터 튜닝	두 스케줄러 간 비교 실험 진행 (가중치 alpha, beta를 바꿔가며 최적점 도출), 결과 데이터 추출
11~12주차	데이터 시각화 및 포스터/보고서 제작	Matplotlib/Seaborn을 활용한 비용 절감 및 지연 시간 그래프 제작, 최종 졸업 전시 포스터 디자인 및 논문 완성